Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

TP2

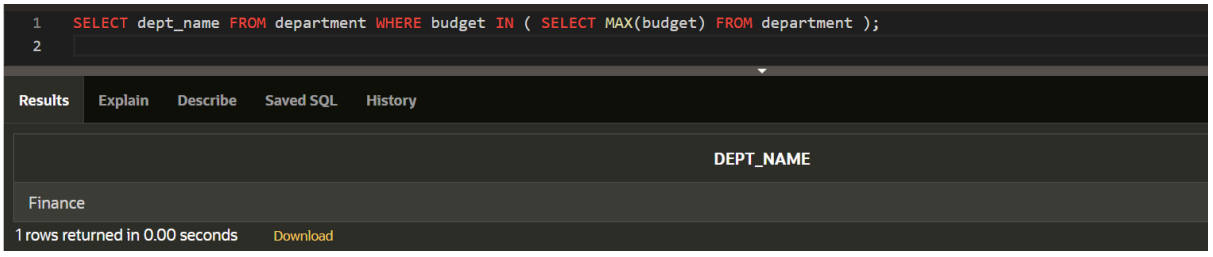
Réalisé par:
BRAHIMI Kaoutar

2024-2025

Exercice 1 :

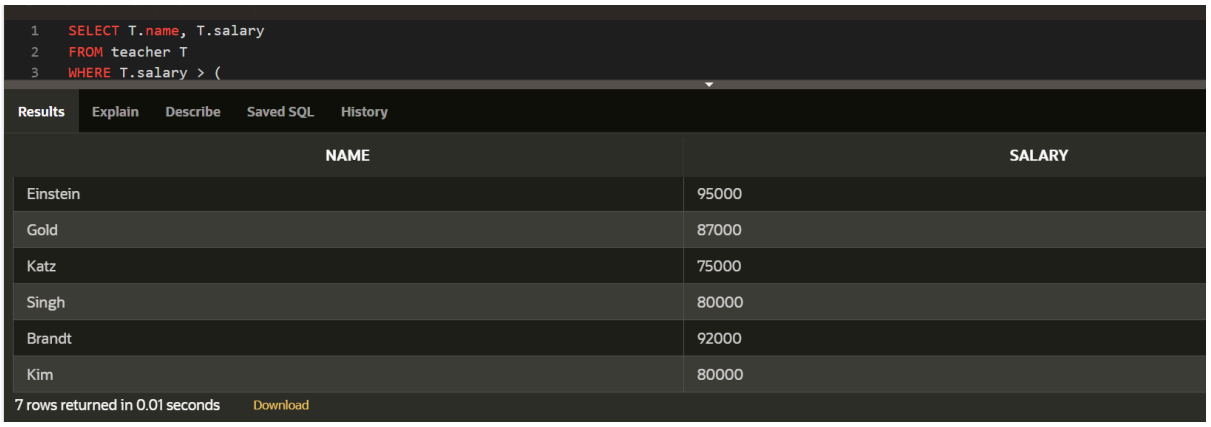
1-

```
SELECT dept_name
FROM department
WHERE budget IN ( SELECT MAX(budget) FROM department );
```



2-

```
SELECT T.name, T.salary
FROM teacher T
WHERE T.salary > (
  SELECT AVG(B.salary)
  FROM teacher B
);
```



3-

```
SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS common_courses
FROM teacher
JOIN teaches ON teacher.ID = teaches.ID
JOIN takes ON takes.course_id = teaches.course_id
        AND takes.sec_id = teaches.sec_id
        AND takes.semester = teaches.semester
        AND takes.year = teaches.year
JOIN student ON student.ID = takes.ID
GROUP BY teacher.name, student.name
HAVING COUNT(*) >= 2;
```

```
SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS common_courses
FROM teacher
JOIN teaches ON teacher.ID = teaches.ID
```

Results | Explain | Describe | Saved SQL | History

TEACHER_NAME	STUDENT_NAME	COMMON_COURSES
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Zhang	2

5 rows returned in 0.05 seconds Download

4-

```
SELECT T.teacher_name, T.student_name, T.total_count
FROM (
   SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS total_count
   FROM teacher
   JOIN teaches ON teacher.ID = teaches.ID
   JOIN takes ON takes.course_id = teaches.course_id
            AND takes.sec_id = teaches.sec_id
            AND takes.semester = teaches.semester
            AND takes.year = teaches.year
   JOIN student ON student.ID = takes.ID
   GROUP BY teacher.name, student.name
) T
WHERE T.total_count >= 2
ORDER BY T.teacher_name;
```

```
SELECT T.teacher_name, T.student_name, T.total_count
FROM (
   SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS total_count
```

Results | Explain | Describe | Saved SQL | History

TEACHER_NAME	STUDENT_NAME	TOTAL_COUNT
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Srinivasan	Zhang	2

5 rows returned in 0.04 seconds Download

5-

```
SELECT ID, name
FROM student
WHERE ID NOT IN (
   SELECT student.ID
```

```
    FROM student
    JOIN takes ON takes.ID = student.ID
    WHERE takes.year < 2009
);
```

```
SELECT ID, name
FROM student
WHERE ID NOT IN (
```

Results | Explain | Describe | Saved SQL | History

ID	NAME
76653	Aoi
23121	Chavez
55739	Sanchez
00128	Zhang
98765	Bourikas

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.02 seconds Download

6-

```
SELECT *
FROM teacher
WHERE name LIKE 'E%';
```

```
SELECT *
FROM teacher
WHERE name LIKE 'E%';
```

Results | Explain | Describe | Saved SQL | History

ID	NAME	DEPT_NAME	SALARY
22222	Einstein	Physics	95000
32343	El Said	History	60000

2 rows returned in 0.00 seconds Download

7-

```
SELECT name
FROM teacher T1
WHERE 3 = (
    SELECT COUNT(DISTINCT T2.salary)
    FROM teacher T2
    WHERE T2.salary > T1.salary
);
```

```
SELECT name
FROM teacher T1
WHERE 3 = (
```

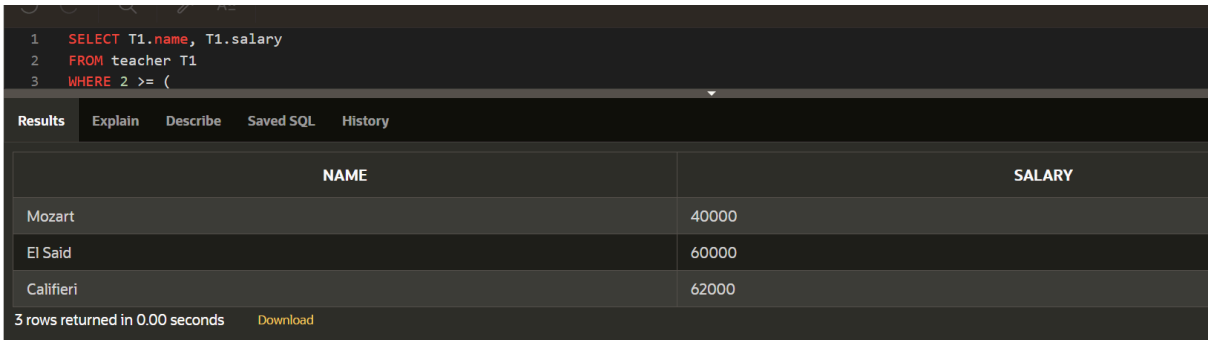
Results | Explain | Describe | Saved SQL | History

NAME
Gold

1 rows returned in 0.01 seconds Download

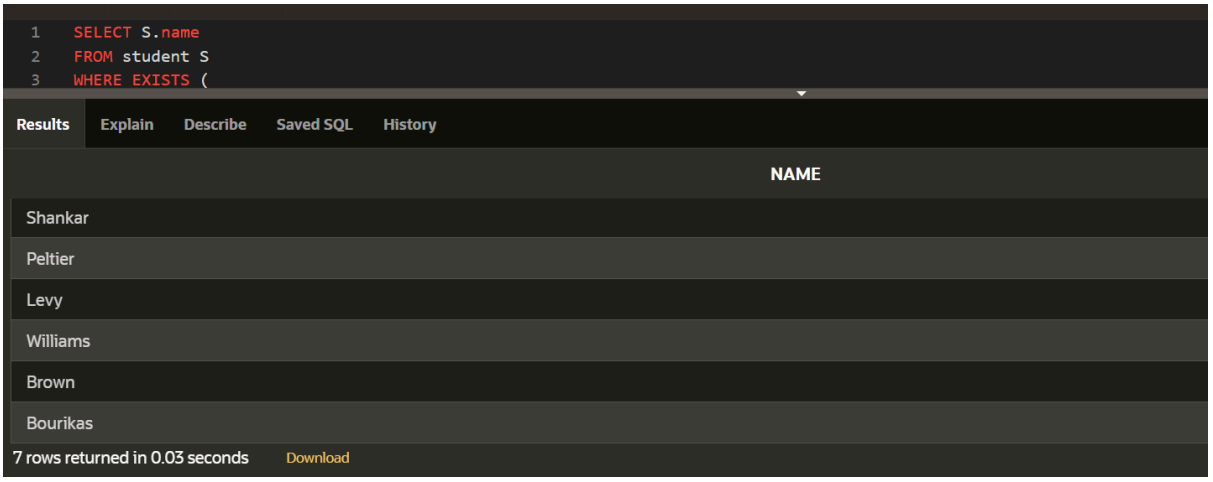
8-

```
SELECT T1.name, T1.salary
FROM teacher T1
WHERE 2 >= (
  SELECT COUNT(DISTINCT T2.salary)
  FROM teacher T2
  WHERE T2.salary < T1.salary
)
ORDER BY T1.salary ASC;
```



9-

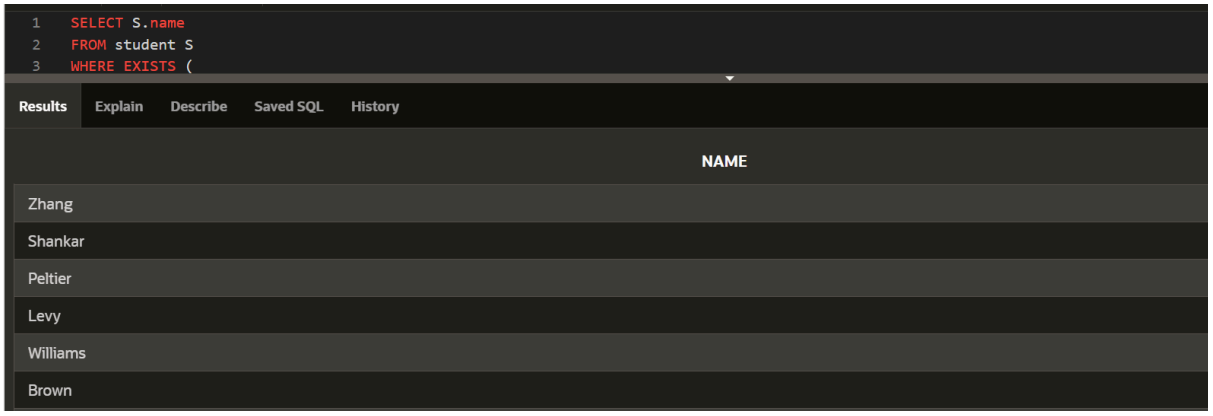
```
SELECT S.name
FROM student S
WHERE EXISTS (
  SELECT 1
  FROM takes
  WHERE takes.ID = S.ID
    AND takes.semester = 'Fall'
    AND takes.year = 2009
);
```



10-

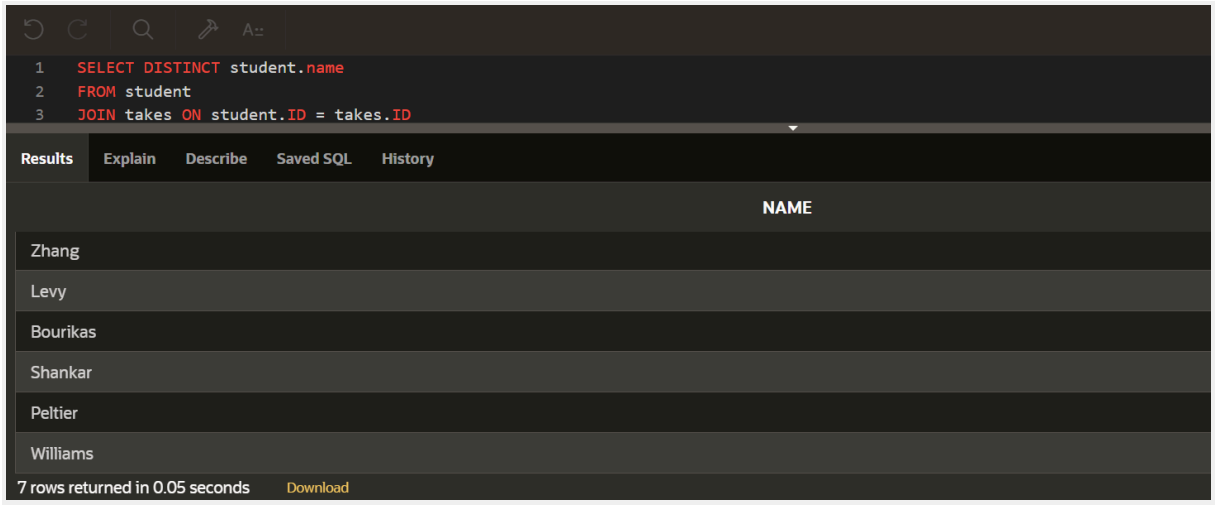
```
SELECT S.name
```

```
FROM student S
        WHERE EXISTS (
          SELECT 1
          FROM takes
          WHERE takes.ID = S.ID
            AND takes.semester = 'Fall'
            AND takes.year = 2009
        );
```



11-

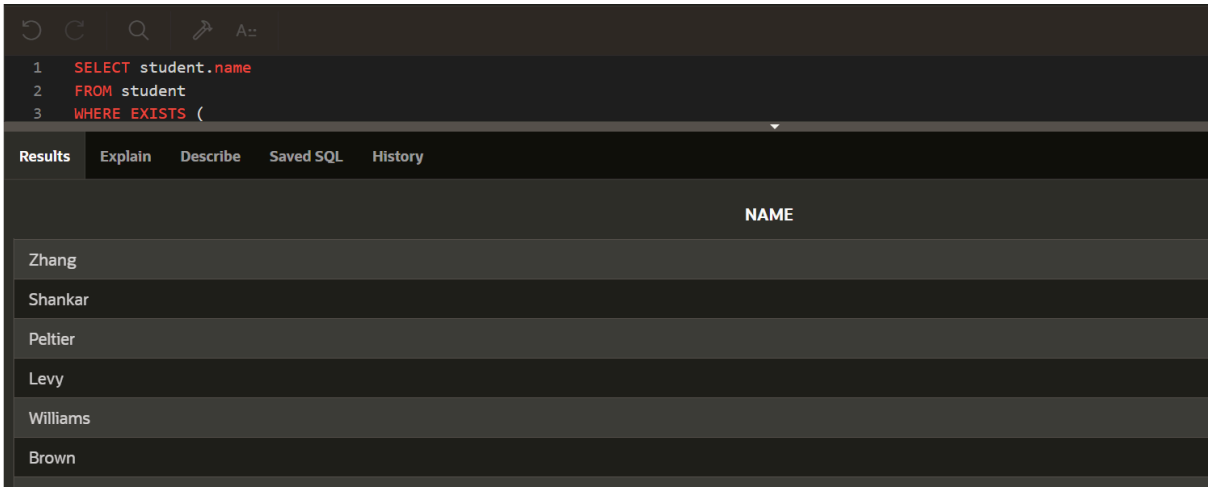
```
SELECT DISTINCT student.name
FROM student
JOIN takes ON student.ID = takes.ID
WHERE takes.semester = 'Fall' AND takes.year = 2009;
```



12-

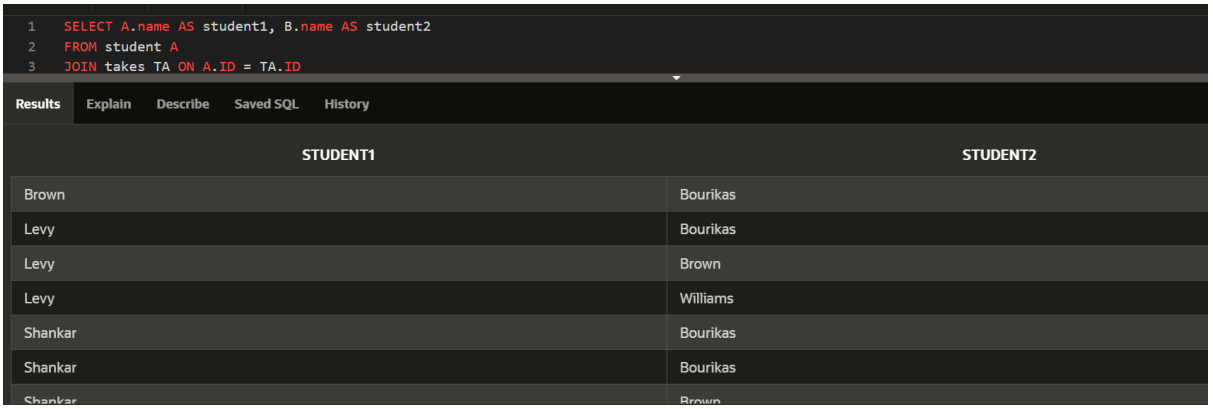
```
SELECT student.name
FROM student
WHERE EXISTS (
  SELECT 1
  FROM takes
```

```
    WHERE takes.ID = student.ID
      AND takes.semester = 'Fall'
      AND takes.year = 2009
);
```



13-

```
SELECT A.name AS student1, B.name AS student2
FROM student A
JOIN takes TA ON A.ID = TA.ID
JOIN takes TB ON TA.course_id = TB.course_id
            AND TA.sec_id = TB.sec_id
            AND TA.semester = TB.semester
            AND TA.year = TB.year
JOIN student B ON B.ID = TB.ID
WHERE A.ID < B.ID
ORDER BY A.name, B.name;
```



Exercice 2 :

Cas 1 : R(A, B, C) avec F = {A → B, B → C}

Dans cette relation, on observe que A détermine B, et B détermine C, ce qui signifie par transitivité que A détermine aussi C. Ainsi, A est une clé candidate, et même la clé primaire.
La relation est en 1NF car chaque champ contient des valeurs indivisibles.
Elle est également en 2NF, étant donné que les attributs non clés (B et C) dépendent entièrement de la clé primaire A.
Cependant, elle n'est pas en 3NF en raison de la dépendance transitive A → B → C. Cela constitue une violation de la 3NF.
De plus, la BCNF n'est pas respectée, car la dépendance B → C ne provient pas d'une super-clé.
Pour normaliser cette relation en 3NF, il faut la décomposer en deux relations :

- **R1(A, B)**

- **R2(B, C)**
  Cette décomposition élimine la dépendance transitive tout en conservant les dépendances fonctionnelles essentielles.

Cas 2 : R(A, B, C) avec F = {A → C, A → B}

Dans ce cas, A détermine à la fois B et C, ce qui en fait la clé primaire de la relation.
La relation est en 1NF car les données sont atomiques.
Elle respecte également la 2NF car chaque attribut dépend totalement de la clé primaire (A).
Aucune dépendance transitive n'existe, donc la 3NF est aussi satisfaite.
Enfin, puisque toutes les dépendances fonctionnelles sont basées sur une super-clé (A), la relation est aussi conforme à la BCNF.
Il n'est donc pas nécessaire de la décomposer.

Cas 3 : R(A, B, C) avec F = {A,B → C ; C → B}

Ici, la combinaison (A, B) détermine C, ce qui en fait la clé primaire. En parallèle, C détermine B, ce qui introduit une dépendance fonctionnelle entre un attribut non clé (C) et un composant de la clé (B).
Cela indique une violation de la 2NF, car B dépend d'une partie seulement de la clé (via C), et donc la 3NF et la BCNF ne sont pas respectées non plus.
Pour corriger cette anomalie et atteindre la 2NF, on peut scinder la relation en deux sous-relations :

- **R1(C, B)**, qui capte la dépendance **C → B**

- **R2(A, C)**, qui conserve la dépendance **A, B → C** sous la forme implicite **A → C** avec la dépendance C → B conservée dans R1.